DATAFORGE 2026 // PATHWAY TRACK // ONE-PAGE CONCEPT SUMMARY (~850 WORDS)

The KV Cache Bottleneck vs. Dragon Hatchling (BDH)

Why Attention Eats GPU Memory, and How Biological Synapses Maintain O(1) Constant Context

> **CENTRAL FALSIFIABLE CLAIM**
> "When sequence length scales from 1k to 128k tokens, Transformer Key-Value (KV) cache memory scales linearly O(N), causing VRAM exhaustion and out-of-memory crashes. In contrast, Dragon Hatchling's (BDH) recurrent synaptic memory maintains a strictly constant O(1) footprint, subject to retrieval degradation (interference noise) due to finite state capacity."

1. The Problem: Autoregressive Decoding & The KV Cache Tax

In standard Transformer self-attention, generating token *t* requires computing dot products with all past keys *1 … t-1*. Discarding past representations would force re-running the entire forward pass across all previous tokens on every step—an intolerable *O(N³)* cumulative compute penalty. Engineers solved this by caching all past Key and Value vectors in GPU High Bandwidth Memory (HBM). While this drops generation compute to *O(1)* per step, it incurs a severe physical memory footprint:

```
Memory_KV = 2 × n_layers × n_kv_heads × d_head × N × B × bytes_per_element
```

For **Llama-3 70B** (80 layers, 8 GQA heads, head dim 128, FP16), each token requires **320 KB** of cache. At 128,000 tokens, a single conversation monopolizes **40.0 GB** of VRAM. With a modest batch size of *B=4* concurrent users, the KV cache alone demands **160.0 GB**—exceeding two entire 80 GB NVIDIA A100 GPUs solely for conversation history, before allocating a single byte for weights.

2. The Mechanism: Softmax Coupling vs. Hebbian Synapses

The root cause of linear memory growth is the **softmax operator**. In $Attention(Q,K,V) = softmax(QK^T / \sqrt{d})\,V$, the exponent and denominator non-linearly couple the current query with all past keys simultaneously. They cannot be mathematically separated or pre-accumulated into a fixed-size running summary.

Pathway's **Dragon Hatchling (BDH)** (*Kosowski et al., arXiv:2509.26507*) replaces external token caching with an internal, fixed-shape synaptic matrix $S_t \in \mathbb{R}^{d \times d}$. As new tokens arrive, connection weights rewire in place via local **Hebbian plasticity**:

$$S_t = \lambda\, S_{t-1} + \eta\, \left(\sigma^{+}(x_t) \otimes \sigma^{+}(x_t) \right) - \gamma\, S_{t-1}$$

Because S_t remains a constant $d \times d$ matrix at every step, sequence length determines only the number of recurrent updates, **never the memory footprint**. A 1B-parameter BDH model requires just **16.77 MB** of state memory whether processing 500 or 5,000,000 tokens—a **2,564× reduction** compared to Llama-3 70B at 128k context.

3. Verified Empirical Benchmark Summary

Context Length (N)	Llama-3 70B (FP16)	Llama-3 8B (FP16)	BDH 1B Recurrent State	Memory Reduction
512 tokens	0.16 GB	0.06 GB	0.016 GB (16 MB)	10×
8,192 tokens	2.50 GB	1.00 GB	0.016 GB (16 MB)	156×
32,768 tokens	10.00 GB	4.00 GB	0.016 GB (16 MB)	625×
65,536 tokens	20.00 GB	8.00 GB	0.016 GB (16 MB)	1,250×
131,072 tokens	40.00 GB	16.00 GB	0.016 GB (16 MB)	**2,564× smaller**

4. The Catch: Rank Bounds & Superposition Noise

While Transformers offer lossless recall by keeping all tokens isolated, BDH compresses unbounded history into a finite matrix. By linear algebra, a matrix of size d has rank at most d. When sequence length $N \gg d$, key-value associations are stored in **superposition**. Cross-talk noise between stored memories scales as $O(\sqrt{N})$, causing older or non-reinforced facts to gradually blur. This establishes the fundamental trade-off: **unbounded lossless memory (Transformer) vs. bounded constant memory (BDH)**.

5. BDH-CQ & The Reasoning Horizon

In **BDH-CQ** (*arXiv:2608.09888*), Pathway extended this principle to reasoning tasks. Instead of generating verbose Chain-of-Thought text tokens that flood the KV cache, BDH-CQ iterates in continuous latent space, achieving **29.5% pass@2** on ARC-AGI-1 at an inference cost of just **$0.0007 per task**. The inevitable future architecture will be **hybrid**: combining a short local attention window for exact syntax with a recurrent biological state for global context.

DataForge 2026 // Pathway Track // Artifact by Anuj & Team Primary literature: arXiv:2509.26507 & arXiv:2608.09888